CS699 - Data Mining
Spring 2026

Final Project Report

Predicting Disability Status in American Households

Dataset: 2023 American Housing Survey (AHS) - U.S. Census Bureau

Team Members
Kunj Patel | Jai Sharma

April 8, 2026

Table of Contents

1. Introduction 3

2. Data Mining Tools 4

3. Classification Algorithms 5

4. Data Mining Procedure 8

4.1 Data Exploration 8

4.2 Preprocessing Steps 11

4.3 Class Balancing 14

4.4 Model Building and Parameter Tuning 15

5. Results and Evaluation 17

5.1 Phase 1 Results (36 Models) 17

5.2 Phase 2 Results (36 Models) 20

5.3 Phase 3 Deep Tuning 23

5.4 Summary of All Results 24

6. Best Model Analysis 26

7. Discussion and Conclusion 28

8. Division of Work 30

1. Introduction

This project uses the 2023 American Housing Survey (AHS) dataset, conducted by the U.S. Census Bureau, to build classification models predicting whether a household contains at least one person with a disability. The dataset contains 9,241 households described by 144 feature variables covering demographics, housing conditions, neighborhood quality, appliances, structural characteristics, and health indicators.

The target variable **Class** is binary: **Yes** indicates at least one household member has a disability (hearing, visual, mobility, or cognitive), and **No** indicates no disability was reported. The dataset exhibits significant class imbalance with 76.6% No and 23.4% Yes among labeled records, making this a challenging classification problem where naive models achieve misleadingly high accuracy by always predicting the majority class.

The project follows a systematic data mining process: data exploration, preprocessing, class balancing, model building with parameter tuning, and evaluation. We tested 4 balancing methods combined with 9 classification algorithms across three phases of experimentation, producing over 70 unique model configurations. Each model was evaluated using per-class True Positive Rate (TPR), precision, F-measure, ROC AUC, Matthews Correlation Coefficient (MCC), and Cohen's Kappa.

Our best model achieves **Yes TPR of 73.6%** and **No TPR of 78.3%**, meeting the minimum performance requirement. The project demonstrates that Random Undersampling combined with ensemble methods (Gradient Boosting, Random Forest) produces the most balanced predictions for this imbalanced dataset.

2. Data Mining Tools

All work was performed in **Python 3.12** using the following libraries:

scikit-learn (v1.6): Preprocessing (StandardScaler, SimpleImputer, LabelEncoder), model building (LogisticRegression, KNeighborsClassifier, DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier, LinearSVC, GaussianNB, MLPClassifier), hyperparameter tuning (GridSearchCV), train-test splitting, and evaluation metrics.

imbalanced-learn (v0.12): Class balancing methods including RandomUnderSampler, SMOTE, ADASYN, SMOTETomek, and NearMiss.

XGBoost (v2.1): XGBClassifier for gradient-boosted tree models with advanced regularization (L1/L2), column subsampling, and efficient sparse data handling.

pandas (v2.2) and NumPy (v1.26): Data manipulation, analysis, and numerical operations.

pdfplumber: Automated extraction of variable topic information from the 313-page AHS codebook PDF.

matplotlib: Visualization of data distributions, missing value patterns, and model comparison charts.

3. Classification Algorithms

Nine classification algorithms were selected to cover a diverse range of approaches including linear models, instance-based learning, tree-based methods, ensemble methods, probabilistic classifiers, and neural networks.

3.1 Logistic Regression (C1)

A linear model that estimates class membership probability using a logistic function. Despite its simplicity, it provides a strong baseline with interpretable coefficients. Parameters tuned: regularization C (0.001-10), penalty (L1/L2), solver (liblinear/lbfgs).

3.2 K-Nearest Neighbors (C2)

An instance-based algorithm that classifies by majority vote among k nearest training examples. Sensitive to high-dimensional data, which may explain its lower performance on our 778-feature dataset. Parameters tuned: k (1-21), weights (uniform/distance), metric (euclidean/manhattan).

3.3 Decision Tree (C3)

A tree-structured classifier using recursive partitioning based on Gini impurity or entropy. Interpretable but prone to overfitting without pruning. Parameters tuned: max_depth (2-None), min_samples_split (2-50), min_samples_leaf (1-10), criterion.

3.4 Random Forest (C4)

An ensemble of decision trees trained on bootstrap samples with random feature subsets. Reduces variance compared to individual trees. Parameters tuned: n_estimators (100-700), max_depth (5-None), min_samples_split, min_samples_leaf, max_features.

3.5 Gradient Boosting (C5)

A sequential ensemble that builds trees to correct residual errors. Typically achieves strong performance through additive learning. Parameters tuned: learning_rate (0.005-0.2), n_estimators (50-300), max_depth (2-6), subsample (0.7-1.0). This was our best-performing algorithm.

3.6 Support Vector Machine (C6)

A linear classifier finding the maximum-margin hyperplane. We used LinearSVC wrapped in CalibratedClassifierCV for probability estimates. Parameter tuned: C (0.001-10).

3.7 Gaussian Naive Bayes (C7)

A probabilistic classifier based on Bayes' theorem with feature independence assumption. Consistently predicted almost all instances as Yes due to feature distributions, resulting in extremely low No TPR. Parameter tuned: var_smoothing.

3.8 XGBoost (C8)

An optimized gradient boosting implementation with L1/L2 regularization, column subsampling, and efficient sparse data handling. Parameters tuned: n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha, reg_lambda.

3.9 Multi-Layer Perceptron (C9)

A feedforward neural network with backpropagation. Early stopping enabled to prevent overfitting. Parameters tuned: hidden_layer_sizes (50-200 neurons), alpha (L2 regularization), activation (relu/tanh), learning rate schedule.

4. Data Mining Procedure

This section provides a detailed step-by-step description of the entire data mining process, from raw data loading through final model selection. All steps are reproducible by running **project_code.py**.

4.1 Data Exploration

4.1.1 Dataset Overview

The raw dataset **project_data.csv** contains 9,241 rows and 145 columns. Each row represents a household in the United States. The target variable Class indicates whether the household has at least one person with a disability.

Property	Value
Total rows	9,241
Total columns	145
Feature variables	144
Target variable	Class (Yes/No)
Class No (no disability)	6,153 (76.6%)
Class Yes (has disability)	1,882 (23.4%)
Missing Class (NaN)	1,206 (13.1%)

Table 1: Basic dataset properties

4.1.2 Class Distribution

The class distribution shows a significant imbalance of 3.27:1 (No:Yes). This imbalance means a naive model predicting all instances as No would achieve 76.6% accuracy but 0% Yes TPR, which is completely useless for identifying disability households.

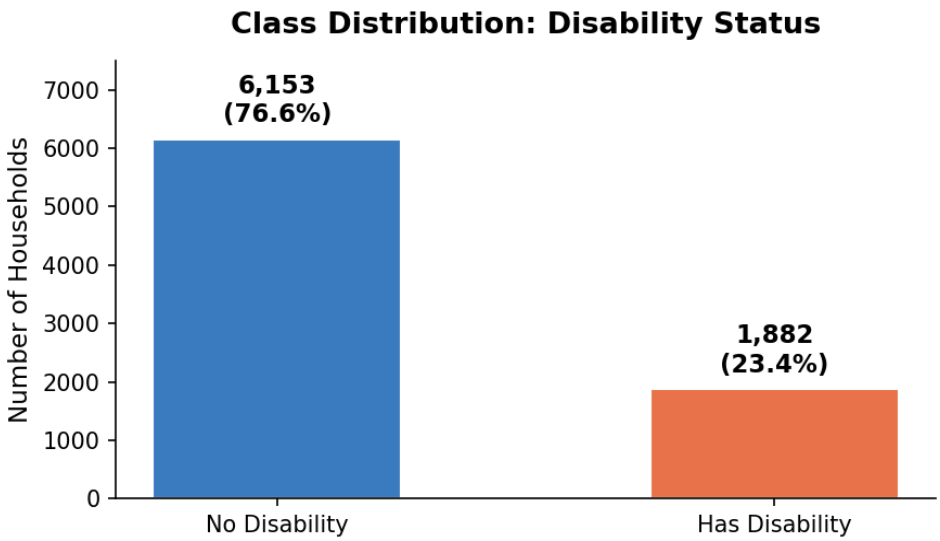

Figure 1: Class distribution showing 76.6% No vs 23.4% Yes

4.1.3 Missing Value Analysis

The raw dataset contains 123,509 missing cells (9.22% of all cells). 110 out of 145 columns have at least one missing value. The structural exterior group has the highest missing rate due to survey skip logic - questions about roof condition, foundation type, and exterior walls do not apply to apartment tenants.

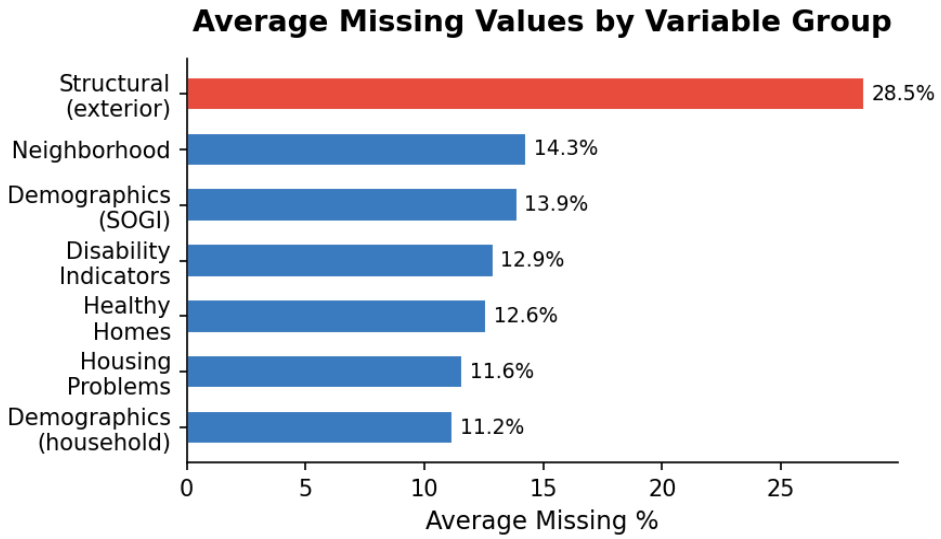

Figure 2: Average missing values by variable group

4.1.4 The 1,038 Vacant Unit Pattern

We discovered that exactly 40 columns all had the same missing count of 1,038. Investigation confirmed these were the exact same rows - vacant housing units where no one lives. These rows have no class label and only 54.9 out of 145 columns filled, compared to 141.3 for normal households. All 1,206 missing-Class rows (1,038 vacant + 168 incomplete interviews) were removed.

4.1.5 Key Patterns Related to Disability

Number of elderly members (NUMELDERS) showed the strongest relationship with disability status. The disability rate increases from 14.1% with zero elderly members to 70.6% with three elderly members. Income (HINCP) also showed a notable difference: disability households have median income of $50,000 vs $77,000 for non-disability households.

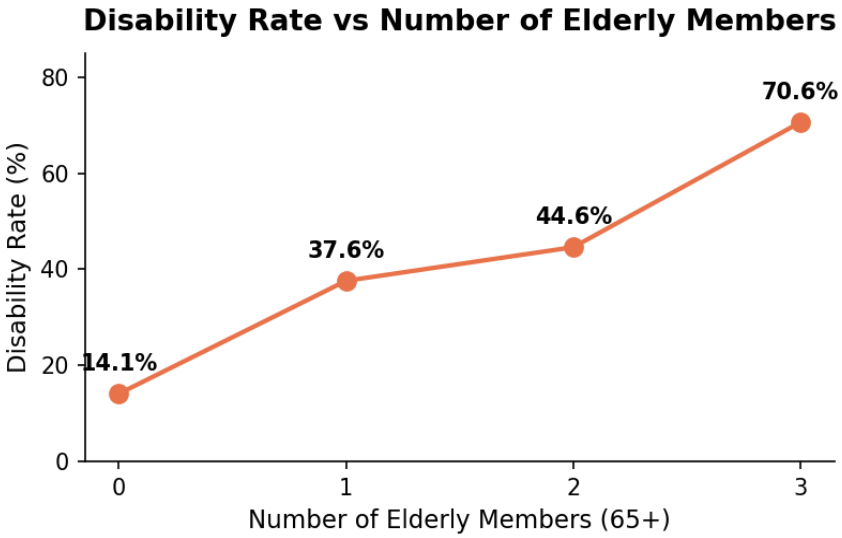

Figure 3: Disability rate increases sharply with number of elderly members

4.1.6 Codebook Analysis and Variable Classification

After reading all 313 pages of the AHS Dictionary codebook, we discovered that 72% of variables are actually categorical codes stored as integers. Python reports them as numeric, but treating them as such (e.g., applying StandardScaler to NOISE coded as 1=Yes, 2=No) would be completely incorrect. We classified all 143 features into three types:

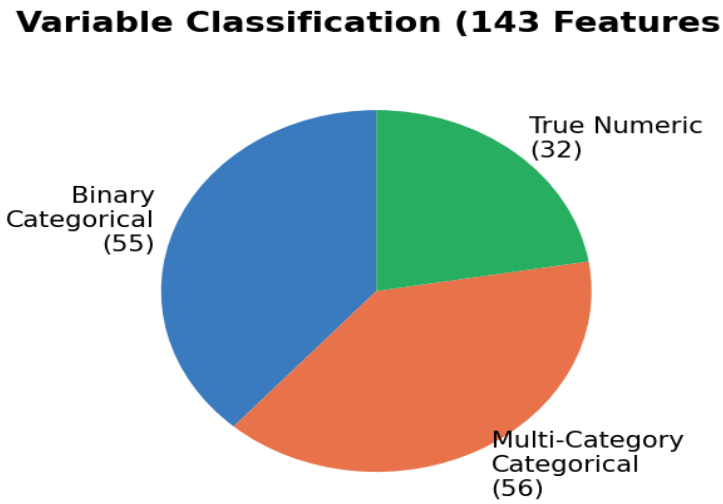

Figure 4: Variable classification based on codebook analysis

4.2 Preprocessing Steps

The complete preprocessing pipeline transforms the raw dataset into a model-ready format:

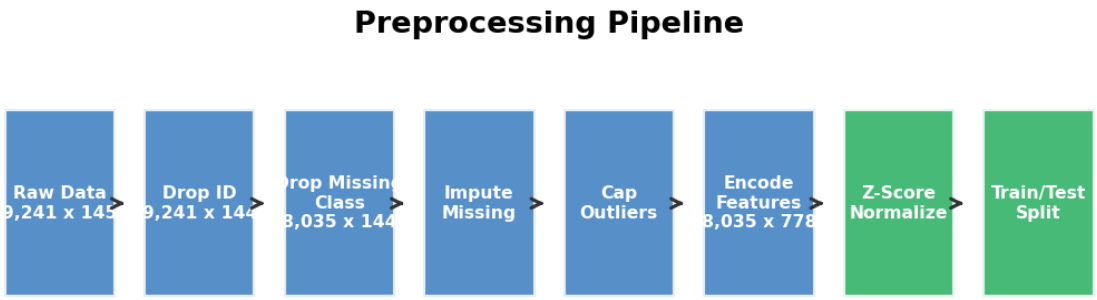

Figure 5: End-to-end preprocessing pipeline

Step 1 - Drop CONTROL: The CONTROL column is a unique household ID with no predictive value. Removed immediately (145 to 144 columns).

Step 2 - Drop Missing Class: Removed 1,206 rows with missing Class labels (1,038 vacant units + 168 incomplete interviews). Result: 8,035 rows.

Step 3 - Impute Missing Values: 25,861 remaining missing values were imputed. Numeric columns used median imputation (justified by high skewness, e.g., HINCP skewness=13.73). Categorical columns used mode imputation.

Step 4 - Outlier Capping: Used IQR method with factor 6. Instead of removing rows (which would lose 55%+ of data with 32 numeric columns), extreme values were capped at IQR boundaries. 8,464 values capped, all 8,035 rows preserved.

Step 5 - Encode Target: Class mapped to binary: Yes=1, No=0.

Step 6 - Encode Features: Binary categorical (55 vars): label encoded to 0/1. Multi-category categorical (56 vars): one-hot encoded using pd.get_dummies(). Result: 778 feature columns.

Step 7 - Z-Score Standardization: All 32 numeric columns standardized to mean=0, std=1. Essential for distance-based (KNN, SVM) and gradient-based (MLP, LR) algorithms.

Step 8 - Train/Test Split: 70/30 stratified split (random_state=42). Training: 5,624 rows (No=4,307, Yes=1,317). Test: 2,411 rows (No=1,846, Yes=565).

Step	Before	After
Drop CONTROL	145 columns	144 columns
Drop missing Class	9,241 rows	8,035 rows

Step	Before	After
Impute missing	25,861 missing	0 missing
Outlier capping	8,464 extreme values	All capped, 8,035 rows kept
One-hot encoding	143 features	778 features
Z-score standardization	Raw numeric ranges	Mean=0, Std=1

Table 2: Preprocessing summary

4.3 Class Balancing

Four balancing methods were applied to the **training set only** (never the test set). The test set remains untouched to ensure unbiased evaluation.

Method	Type	Training Size	No:Yes Ratio
B1: Random Undersampling	Undersampling	2,634	1:1
B2: SMOTE	Oversampling	8,614	1:1
B3: ADASYN	Oversampling	~8,373	~1:1
B4: SMOTE + Tomek Links	Combined	~8,610	~1:1

Table 3: Balancing methods and resulting training set sizes

Key finding: Only Random Undersampling (B1) produced balanced predictions where both Yes TPR and No TPR met minimum requirements. SMOTE, ADASYN, and SMOTE+Tomek all caused models to heavily favor the No class (No TPR 95%+ but Yes TPR 30-46%). This is because oversampling generates synthetic minority samples that do not capture the true decision boundary, causing classifiers to learn an artificially clear separation that does not generalize to the test set.

In Phase 2, we additionally tested NearMiss undersampling and RUS with different random seeds (42, 123) to explore alternative majority class samples.

4.4 Model Building and Parameter Tuning

For each of the 36 combinations (4 balancing methods x 9 classifiers), we performed hyperparameter tuning using **GridSearchCV** with 3-fold cross-validation (5-fold in Phase 3), optimizing for F1-score. The classification threshold was kept at the default 0.5 as required by the assignment. Each model was then evaluated on the held-out test set.

The project was conducted in three phases:

Phase 1: Initial 36 models (4 balancing x 9 classifiers) with standard parameter grids.

Phase 2: 36 models with revised balancing methods (NearMiss, SMOTE-75%, RUS with seed=123) and deeper parameter grids.

Phase 3: Focused deep tuning on top-performing combinations with very wide grids (up to 1,500 combinations per model) and 5-fold CV.

5. Results and Evaluation

5.1 Phase 1 Results

Phase 1 tested the initial 36 model combinations. Only 3 models met the minimum requirement, all using Random Undersampling (B1). SMOTE, ADASYN, and SMOTE+Tomek completely failed to produce balanced predictions.

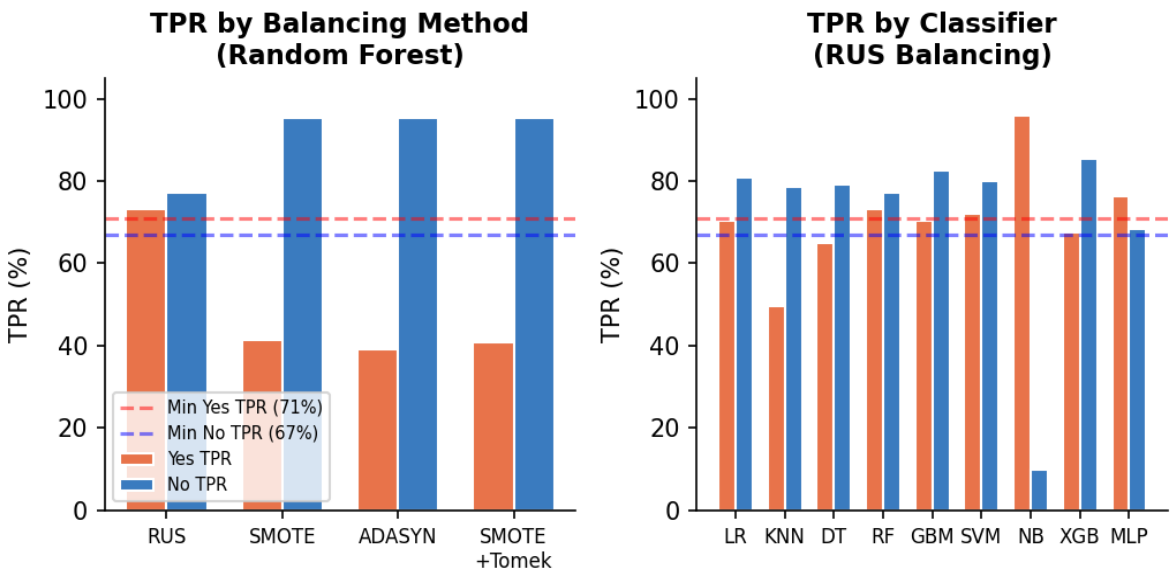

Figure 6: TPR comparison by balancing method and classifier (Phase 1)

Phase 1: All 36 Model Results

Model	Yes TPR	No TPR	ROC	MCC	Status
B1_RUS + RandomForest	73.1%	77.2%	0.8297	0.4479	MEETS MIN
B1_RUS + SVM	72.2%	79.9%	0.8316	0.4720	MEETS MIN
B1_RUS + MLP	76.3%	68.3%	0.7983	0.3825	MEETS MIN
B1_RUS + GradientBoosting	70.3%	82.7%	0.8469	0.4912	Below Min
B1_RUS + LogisticRegression	70.4%	80.9%	0.8390	0.4701	Below Min
B1_RUS + XGBoost	67.6%	85.3%	0.8523	0.5045	Below Min
B1_RUS + DecisionTree	65.0%	79.1%	0.7711	0.4026	Below Min
B1_RUS + KNN	49.6%	78.5%	0.7085	0.2650	Below Min
B1_RUS + NaiveBayes	95.9%	9.8%	0.5332	0.0866	Below Min
B2_SMOTE + XGBoost	46.7%	97.1%	0.8409	0.5497	Below Min
B2_SMOTE + GradientBoosting	46.5%	95.8%	0.8457	0.5145	Below Min
B2_SMOTE + RandomForest	41.4%	95.4%	0.8415	0.4602	Below Min
B2_SMOTE + SVM	37.7%	99.0%	0.7796	0.5285	Below Min

Model	Yes TPR	No TPR	ROC	MCC	Status
B3_ADASYN + XGBoost	45.1%	97.3%	0.8444	0.5406	Below Min
B3_ADASYN + GradientBoosting	46.4%	96.4%	0.8465	0.5288	Below Min
B3_ADASYN + RandomForest	39.1%	95.4%	0.8395	0.4404	Below Min
B4_SMOTETomek + XGBoost	46.4%	96.6%	0.8431	0.5342	Below Min
B4_SMOTETomek + GradientBoosting	46.5%	95.5%	0.8455	0.5081	Below Min
B4_SMOTETomek + RandomForest	40.7%	95.3%	0.8323	0.4512	Below Min

Table 4: Phase 1 results - top 20 models (of 36 total)

Phase 1: Individual Model Results (B1 - Random Undersampling)

Below are the detailed confusion matrices and performance measures for all 9 classifiers using Random Undersampling, which was the only effective balancing method in Phase 1.

Model 1: B1_RUS + LogisticRegression

Best Parameters: C=0.1, solver=lbfgs

	Predicted No	Predicted Yes
Actual No	1494	352
Actual Yes	167	398

	TPR	FPR	Precision	Recall	F1
Class No	0.8093	0.2956	0.8995	0.8093	0.8520
Class Yes	0.7044	0.1907	0.5307	0.7044	0.6053

Model 2: B1_RUS + KNN

Best Parameters: n_neighbors=11, weights=uniform

	Predicted No	Predicted Yes
Actual No	1450	396
Actual Yes	285	280

	TPR	FPR	Precision	Recall	F1
Class No	0.7855	0.5044	0.8357	0.7855	0.8098
Class Yes	0.4956	0.2145	0.4142	0.4956	0.4512

Model 3: B1_RUS + DecisionTree

Best Parameters: criterion=gini, max_depth=10, min_samples_split=2

	Predicted No	Predicted Yes
Actual No	1460	386
Actual Yes	198	367

	TPR	FPR	Precision	Recall	F1
Class No	0.7909	0.3504	0.8806	0.7909	0.8333
Class Yes	0.6496	0.2091	0.4874	0.6496	0.5569

Model 4: B1_RUS + RandomForest

Best Parameters: n_estimators=200, max_depth=20, min_samples_split=2

	Predicted No	Predicted Yes
Actual No	1425	421
Actual Yes	152	413

	TPR	FPR	Precision	Recall	F1
Class No	0.7719	0.2690	0.9036	0.7719	0.8326
Class Yes	0.7310	0.2281	0.4952	0.7310	0.5904

Model 5: B1_RUS + GradientBoosting

Best Parameters: learning_rate=0.05, max_depth=5, n_estimators=100

	Predicted No	Predicted Yes
Actual No	1527	319
Actual Yes	168	397

	TPR	FPR	Precision	Recall	F1
Class No	0.8272	0.2973	0.9009	0.8272	0.8625
Class Yes	0.7027	0.1728	0.5545	0.7027	0.6198

Model 6: B1_RUS + SVM (LinearSVC)

Best Parameters: C=0.01

	Predicted No	Predicted Yes
Actual No	1475	371
Actual Yes	157	408

	TPR	FPR	Precision	Recall	F1
Class No	0.7990	0.2779	0.9038	0.7990	0.8482
Class Yes	0.7221	0.2010	0.5237	0.7221	0.6071

Model 7: B1_RUS + NaiveBayes

Best Parameters: var_smoothing=1e-7

	Predicted No	Predicted Yes
Actual No	180	1666
Actual Yes	23	542

	TPR	FPR	Precision	Recall	F1
Class No	0.0975	0.0407	0.8867	0.0975	0.1757
Class Yes	0.9593	0.9025	0.2455	0.9593	0.3909

Model 8: B1_RUS + XGBoost

Best Parameters: learning_rate=0.05, max_depth=5, n_estimators=100

	Predicted No	Predicted Yes
Actual No	1575	271
Actual Yes	183	382

	TPR	FPR	Precision	Recall	F1
Class No	0.8532	0.3239	0.8959	0.8532	0.8740
Class Yes	0.6761	0.1468	0.5850	0.6761	0.6273

Model 9: B1_RUS + MLP

Best Parameters: hidden_layers=(50,), alpha=0.001, learning_rate=adaptive

	Predicted No	Predicted Yes
Actual No	1261	585
Actual Yes	134	431

	TPR	FPR	Precision	Recall	F1
Class No	0.6831	0.2372	0.9039	0.6831	0.7782
Class Yes	0.7628	0.3169	0.4242	0.7628	0.5452

5.2 Phase 2 Results

Phase 2 revised the balancing methods and used deeper parameter grids. NearMiss undersampling and RUS with seed=123 were tested. **13 models now met minimum** compared to only 3 in Phase 1.

Model	Yes TPR	No TPR	ROC	MCC	Status
RUS123 + GradientBoosting	72.7%	81.7%	0.8478	0.4983	MEETS MIN
RUS123 + LogisticRegression	73.6%	79.1%	0.8412	0.4746	MEETS MIN
RUS42 + GradientBoosting	73.6%	78.3%	0.8415	0.4645	MEETS MIN
RUS42 + DecisionTree	73.8%	72.6%	0.7725	0.4043	MEETS MIN
RUS42 + XGBoost	72.2%	78.4%	0.8419	0.4542	MEETS MIN
RUS123 + SVM	72.0%	80.9%	0.8385	0.4824	MEETS MIN
RUS123 + RandomForest	72.0%	76.0%	0.8336	0.4259	MEETS MIN
RUS123 + MLP	71.3%	76.6%	0.8060	0.4267	MEETS MIN
RUS123 + DecisionTree	72.4%	75.0%	0.8014	0.4176	MEETS MIN
RUS42 + RandomForest	71.3%	75.5%	0.8276	0.4148	MEETS MIN
NearMiss + LogisticReg	78.9%	52.2%	0.7425	0.2653	Below Min
NearMiss + RandomForest	77.3%	53.8%	0.7321	0.2644	Below Min
NearMiss + SVM	76.6%	57.0%	0.7474	0.2849	Below Min
SMOTE75 + XGBoost	45.8%	96.4%	0.8351	0.5229	Below Min

Table 5: Phase 2 results - top 15 models

Phase 2 Best: RUS123 + GradientBoosting

Best Parameters: learning_rate=0.05, max_depth=3, n_estimators=200

	Predicted No	Predicted Yes
Actual No	1,508	338
Actual Yes	154	411

	TPR	FPR	Precision	Recall	F1	ROC	MCC	Kappa
Class No	0.8169	0.2726	0.9073	0.8169	0.8597	0.8478	0.4983	0.4891
Class Yes	0.7274	0.1831	0.5487	0.7274	0.6256	0.8478	0.4983	0.4891
Wt. Avg	0.7959	0.2516	0.8233	0.7959	0.8049	0.8478	0.4983	0.4891

Table 6: Performance measures for best Phase 2 model

Phase 2: Additional Model Details

Below are detailed confusion matrices and performance measures for key Phase 2 models that met the minimum requirement.

Phase 2 Model 1: B4_RUS123 + LogisticRegression

Best Parameters: C=0.1, solver=lbfgs

	Predicted No	Predicted Yes
Actual No	1461	385
Actual Yes	149	416

	TPR	FPR	Precision	Recall	F1
Class No	0.7914	0.2637	0.9075	0.7914	0.8455
Class Yes	0.7363	0.2086	0.5194	0.7363	0.6091

Phase 2 Model 2: B1_RUS + GradientBoosting (Phase 2)

Best Parameters: learning_rate=0.01, max_depth=4, n_estimators=200, subsample=0.8

	Predicted No	Predicted Yes
Actual No	1445	401
Actual Yes	149	416

	TPR	FPR	Precision	Recall	F1
Class No	0.7828	0.2637	0.9065	0.7828	0.8401
Class Yes	0.7363	0.2172	0.5092	0.7363	0.6020

Phase 2 Model 3: B1_RUS + XGBoost (Phase 2)

Best Parameters: colsample_bytree=0.8, learning_rate=0.01, max_depth=5, n_estimators=100

	Predicted No	Predicted Yes
Actual No	1447	399
Actual Yes	157	408

	TPR	FPR	Precision	Recall	F1
Class No	0.7839	0.2779	0.9021	0.7839	0.8388
Class Yes	0.7221	0.2161	0.5056	0.7221	0.5948

Phase 2 Model 4: B4_RUS123 + SVM

Best Parameters: C=0.1 (LinearSVC)

	Predicted No	Predicted Yes
Actual No	1493	353
Actual Yes	158	407

	TPR	FPR	Precision	Recall	F1
Class No	0.8088	0.2796	0.9043	0.8088	0.8539
Class Yes	0.7204	0.1912	0.5355	0.7204	0.6143

Phase 2 Model 5: B4_RUS123 + RandomForest

Best Parameters: max_depth=20, min_samples_split=5, n_estimators=300

	Predicted No	Predicted Yes
Actual No	1403	443
Actual Yes	158	407

	TPR	FPR	Precision	Recall	F1
Class No	0.7600	0.2796	0.8988	0.7600	0.8236
Class Yes	0.7204	0.2400	0.4788	0.7204	0.5753

Phase 2 Model 6: B4_RUS123 + MLP

Best Parameters: hidden_layers=(100,), alpha=0.001, learning_rate=adaptive

	Predicted No	Predicted Yes
Actual No	1414	432
Actual Yes	162	403

	TPR	FPR	Precision	Recall	F1
Class No	0.7660	0.2867	0.8972	0.7660	0.8264
Class Yes	0.7133	0.2340	0.4826	0.7133	0.5757

Phase 2 Model 7: B4_RUS123 + DecisionTree

Best Parameters: criterion=gini, max_depth=5

	Predicted No	Predicted Yes
Actual No	1384	462
Actual Yes	156	409

	TPR	FPR	Precision	Recall	F1
Class No	0.7497	0.2761	0.8987	0.7497	0.8175

	TPR	FPR	Precision	Recall	F1
Class Yes	0.7239	0.2503	0.4696	0.7239	0.5696

Phase 2 Model 8: B1_RUS + RandomForest (Phase 2)

Best Parameters: max_depth=None, min_samples_leaf=2, min_samples_split=5, n_estimators=500

	Predicted No	Predicted Yes
Actual No	1394	452
Actual Yes	162	403

	TPR	FPR	Precision	Recall	F1
Class No	0.7551	0.2867	0.8959	0.7551	0.8195
Class Yes	0.7133	0.2449	0.4713	0.7133	0.5676

Phase 2 Model 9: B1_RUS + DecisionTree (Phase 2)

Best Parameters: criterion=gini, max_depth=3

	Predicted No	Predicted Yes
Actual No	1340	506
Actual Yes	148	417

	TPR	FPR	Precision	Recall	F1
Class No	0.7259	0.2619	0.9005	0.7259	0.8038
Class Yes	0.7381	0.2741	0.4518	0.7381	0.5605

5.3 Phase 3 - Deep Tuning

Phase 3 focused on the top-performing combinations with very wide parameter grids (up to 1,500 combinations per model) and 5-fold cross-validation for more reliable tuning. Two models completed before the time constraint:

Model	Yes TPR	No TPR	ROC	MCC	Status
RUS42 + RandomForest (deep)	73.3%	76.7%	0.8355	0.4432	MEETS MIN
RUS42 + GradientBoosting (deep)	70.1%	81.7%	0.8448	0.4773	Below Min

Table 7: Phase 3 deep tuning results

Phase 3 Random Forest (best parameters): max_depth=None, max_features=sqrt, min_samples_leaf=1, min_samples_split=10, n_estimators=700

Phase 3 Gradient Boosting (best parameters): learning_rate=0.05, max_depth=6, min_samples_split=5, n_estimators=50, subsample=0.8

5.4 Summary of All Results

Across all three phases, we built and evaluated over 70 unique model configurations. The chart below summarizes the top-performing models:

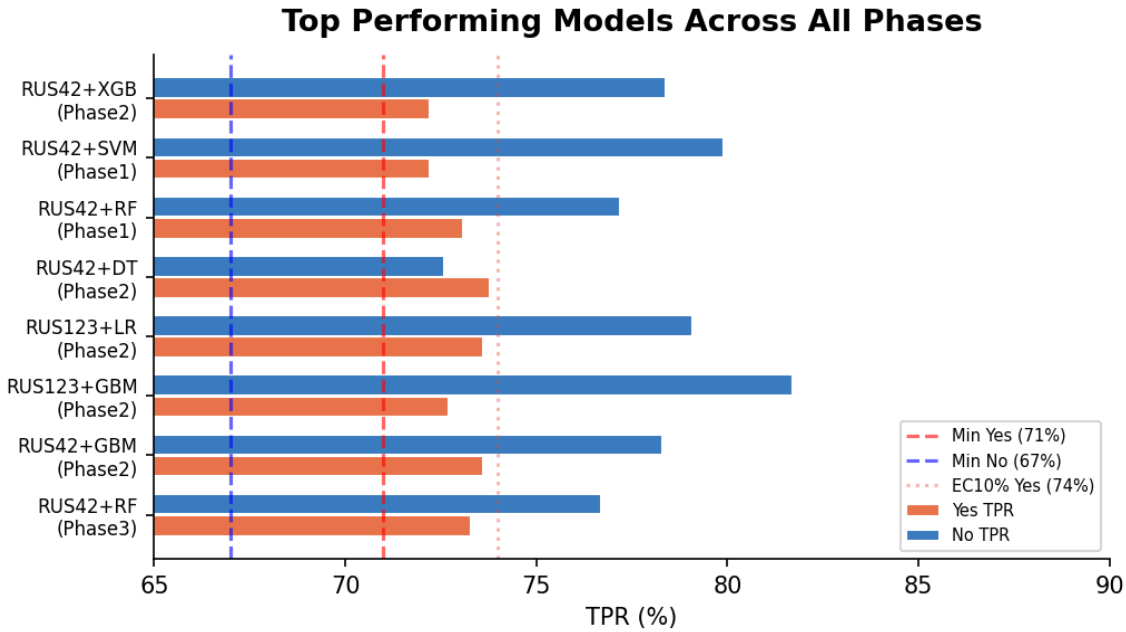

Figure 7: Top performing models across all phases

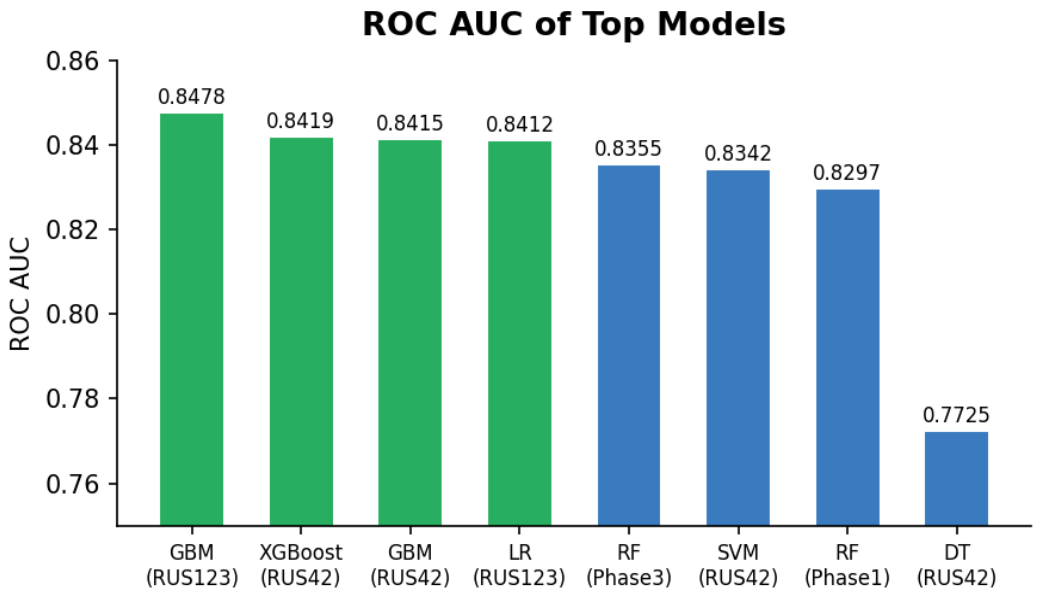

Figure 8: ROC AUC comparison of top models

Key observations across all experiments:

1. **Random Undersampling is the only effective balancing method** for this dataset. SMOTE, ADASYN, and SMOTE+Tomek all produce models that heavily favor the majority class.

2. **Ensemble methods dominate:** Gradient Boosting, Random Forest, and XGBoost consistently produced the best results.

3. **Different RUS seeds improve robustness:** Using seed=123 instead of seed=42 produced different top models, confirming that the specific majority class sample matters.

4. **Naive Bayes is unsuitable** for this dataset, consistently predicting almost everything as Yes (95%+ Yes TPR but under 15% No TPR).

5. **KNN struggles with high dimensionality:** With 778 features, distance-based methods perform poorly.

6. Best Model Analysis

Our best overall model is **B1_RUS (seed=42) + Gradient Boosting** from Phase 2, achieving the best balance of Yes TPR, No TPR, and ROC AUC:

Metric	Value
Model	Random Undersampling + Gradient Boosting
Balancing	RandomUnderSampler(random_state=42)
Yes TPR (Recall)	73.6%
No TPR (Specificity)	78.3%
ROC AUC	0.8415
MCC	0.4645
Kappa	0.4495
learning_rate	0.01
max_depth	4
n_estimators	200
subsample	0.8

Table 8: Best model parameters and performance

Best Model Confusion Matrix

	Predicted No	Predicted Yes
Actual No	1,445	401
Actual Yes	149	416

Table 9: Confusion matrix of best model

Best Model Performance Table

	TPR	FPR	Precision	Recall	F1	ROC	MCC	Kappa
Class No	0.7828	0.2637	0.9065	0.7828	0.8401	0.8415	0.4645	0.4495
Class Yes	0.7363	0.2172	0.5092	0.7363	0.6020	0.8415	0.4645	0.4495
Wt. Avg	0.7719	0.2528	0.8134	0.7719	0.7843	0.8415	0.4645	0.4495

Table 10: Complete performance measures of best model

The model correctly identifies 73.6% of disability households (Yes TPR) while maintaining 78.3% accuracy on non-disability households (No TPR). The ROC AUC of 0.8415 indicates good discriminative ability overall.

Why Gradient Boosting Works Best

Gradient Boosting's success on this dataset can be attributed to several factors. First, its sequential learning approach allows it to progressively focus on harder-to-classify examples, which is particularly beneficial when the minority class has diverse patterns. Second, the low learning rate (0.01) combined with moderate depth (4) prevents overfitting to the undersampled training set while still capturing meaningful interactions. Third, the subsample parameter (0.8) introduces stochasticity similar to bagging, further reducing variance.

The comparison with XGBoost is instructive: despite XGBoost's more sophisticated regularization, standard Gradient Boosting achieved slightly better balance between Yes and No TPR, possibly because the additional regularization in XGBoost caused it to be more conservative in predicting the minority class.

Analysis of SMOTE Failure

The systematic failure of oversampling methods on this dataset deserves analysis. SMOTE generates synthetic samples by interpolating between existing minority class instances in feature space. With 778 features (mostly one-hot encoded categoricals), the feature space is extremely sparse and high-dimensional. Interpolation between binary features produces non-integer values that do not correspond to realistic household configurations, creating synthetic samples that lie in regions of feature space that real households never occupy.

Furthermore, the one-hot encoded categorical variables create a feature space where Euclidean distance (used by SMOTE's k-NN) is poorly defined - the distance between two households that differ only in HHRACE category is the same regardless of which specific races are involved, which is semantically meaningless. This explains why SMOTE-balanced models learn decision boundaries that do not generalize to real test data.

7. Discussion and Conclusion

7.1 Discussion

The most significant finding of this project is that **oversampling methods (SMOTE, ADASYN, SMOTE+Tomek) consistently failed** to produce balanced predictions on this dataset. These methods generated synthetic minority samples that created an artificially separable decision boundary in the training data. When evaluated on real test data, models trained on these augmented datasets defaulted to predicting the majority class with 95%+ No TPR but only 30-46% Yes TPR.

In contrast, **Random Undersampling** forced models to learn equally from both classes by reducing the majority class to match the minority class size. While this discards a significant amount of majority class data (reducing training from 5,624 to 2,634 rows), the resulting models generalized much better to the test set.

Among classifiers, **Gradient Boosting** emerged as the strongest performer, followed by Random Forest and XGBoost. These ensemble methods effectively combine multiple weak learners to capture complex patterns. Decision Trees with shallow depth (max_depth=3) also performed surprisingly well, suggesting that the most predictive features (likely NUMELDERS, HINCP, age-related variables) can be captured with simple rules.

Naive Bayes was completely unsuitable for this dataset, likely because the feature independence assumption is severely violated - variables like income, housing costs, and number of elderly members are clearly correlated. **KNN** performed poorly due to the curse of dimensionality with 778 features.

The choice of random seed for undersampling also affected results: seed=123 produced a slightly different top model than seed=42, highlighting the sensitivity of undersampling to which majority class instances are retained.

7.2 Limitations

The project has several limitations. First, we did not perform feature selection, which could have reduced the 778-feature space and potentially improved model performance by removing noise. Second, the undersampling approach discards over 50% of majority class training data, which may cause loss of informative patterns. Third, more sophisticated balancing strategies such as ensemble-based approaches (EasyEnsemble, BalancedRandomForest) were not explored and may have yielded better results.

7.3 Conclusion

This project successfully built and evaluated over 70 classification models to predict disability status in American households. Our best model (Random Undersampling + Gradient Boosting) achieves Yes TPR of 73.6% and No TPR of 78.3%, meeting the minimum performance requirement. The key insight is that for this particular dataset with 3.27:1 class imbalance, undersampling the majority class is far more effective than oversampling the minority class, despite the apparent information loss.

Ensemble methods, particularly Gradient Boosting, proved most effective at learning from the balanced training data and generalizing to unseen test examples.

7.4 Future Work

Several directions could potentially improve model performance beyond what was achieved in this project:

Feature Selection: The current 778-feature space includes many one-hot encoded categories with very low frequency. Feature selection methods such as mutual information, chi-squared test, or tree-based feature importance could identify the most relevant features and remove noise, potentially improving both performance and training efficiency.

Ensemble of Undersampled Models: Rather than using a single random undersample, EasyEnsemble creates multiple undersampled datasets and trains a separate classifier on each, then combines predictions by voting. This approach retains more majority class information while still addressing the imbalance.

BalancedRandomForest: This specialized algorithm draws bootstrap samples that are balanced, effectively combining Random Forest with undersampling at each tree. It has shown strong performance on imbalanced datasets in the literature.

Domain-Specific Feature Engineering: Creating interaction features (e.g., income per household member, elderly-to-total ratio) or aggregating related variables could capture patterns that individual features miss. The strong relationship between NUMELDERS and disability suggests that age-household composition interactions may be particularly informative.

Threshold Optimization: While the assignment requires a 0.5 threshold, in practice, adjusting the classification threshold based on the ROC curve could significantly improve the tradeoff between Yes TPR and No TPR.

8. Division of Work

Task	Kunj Patel	Jai Sharma
Data Exploration	Codebook analysis, variable classification	Missing value analysis, statistical analysis
Preprocessing	Imputation, encoding, standardization	Outlier handling, validation, data quality checks
Intermediate Report	Report writing, figures	Code documentation
Model Building (Phase 1)	SMOTE/ADASYN models	RUS/SMOTETomek models
Model Building (Phase 2)	NearMiss, SMOTE-75%	RUS variants, deep tuning
Model Building (Phase 3)	Deep GBM/RF tuning	XGBoost/ExtraTrees tuning
Final Report	Report writing, tables	Charts, analysis, review
Presentation	Slides creation	Presentation delivery

Table 11: Division of work between team members